# Shortest-Path Planning in a Large-Scale Road Network: Dijkstra vs A*

Shlok Bothra

Admission number: 25JE1160

## 1. Summary

This report compares Dijkstra's algorithm and A* on connected subgraphs of a real road network: the Great Lakes region (LKS) of the 9th DIMACS Implementation Challenge road data. Both algorithms were implemented from scratch in C++ on an adjacency-list graph with a binary min-heap priority queue, and no library shortest-path routine is used anywhere in the submitted code. Experiments ran on six graph sizes (1,000 to 100,000 vertices) with ten source-destination pairs per size.

The main findings are:

- Dijkstra and A* returned identical shortest-path costs on all 60 source-destination pairs (10 per size, 6 sizes), and every pair was reachable.
- A* explored 66.8% to 77.3% fewer vertices than Dijkstra on average at each size (about 72.6% fewer across the six sizes).
- Execution times ranged from 0.02 ms to 9.67 ms per query, and A* was faster at every size: it needed 26% to 53% of Dijkstra's time (1.9 to 3.8 times faster), and was faster on 53 of the 60 individual queries.
- The A* heuristic scale was between 8.02 and 9.92 depending on the subgraph, against a median weight-to-haversine ratio of 9.99. This ratio sets how much A* can prune while staying optimal.

## 2. Dataset and Graph Construction (Parts A and B)

| Item | Details |
|---|---|
| Dataset | USA road network, Great Lakes region (LKS), distance graph |
| Source | 9th DIMACS Implementation Challenge, Shortest Paths (2005-2006), derived from US Census Bureau TIGER/Line data: http://www.diag.uniroma1.it/challenge9/download.shtml |
| Files used | USA-road-d.LKS.gr.gz (arcs and distance weights) and USA-road-d.LKS.co.gz (node coordinates) |
| Full size | 2,758,119 vertices, 6,885,658 directed arcs (6,794,808 after merging 90,850 parallel arcs) |
| Vertex | A road intersection or road endpoint |
| Edge | A road segment between two vertices, stored as two directed arcs (one per direction) |
| Edge weight | Road distance from the distance graph, a positive integer; about 0.1 m per unit (median weight-to-haversine ratio 9.99) |

The raw files were converted into graphs in these steps:

1. Parsed the .gr file (lines of the form a u v w are directed arcs) and the .co file (coordinates in units of $10^{-6}$ degrees, divided by $10^{6}$ to give degrees). DIMACS node ids are 1-indexed and were shifted to 0-indexed.
2. Cleaned the arcs: self-loops and non-positive weights were removed (none occurred), and parallel arcs were merged by keeping the minimum weight (90,850 arcs merged).

3. Kept the largest connected component. The full LKS graph is a single connected component, so no vertex was removed.
4. Cut out subgraphs of 1,000, 5,000, 10,000, 25,000, 50,000 and 100,000 vertices. Each is the first N vertices reached by a breadth-first search from a fixed seed vertex (original DIMACS id 246165, chosen with random seed 42), keeping every arc whose two endpoints are inside the set. A BFS ball is always connected, and because the same seed vertex is used for every size the subgraphs are nested.
5. Renumbered the vertices 0 to N-1 in BFS order and wrote each subgraph to CSV files (edges: u,v,w; nodes: id, original id, latitude, longitude). The C++ programs load these files into an adjacency list, vector of vector of (to, weight).

Only the information needed for shortest paths was kept: the vertices, the weighted arcs, the coordinates (used by A*) and the id mapping back to the original dataset.

**Graph statistics.** Edges are undirected road segments (arcs divided by 2, because every arc has a matching reverse arc of equal weight). Average degree is arcs divided by vertices. The values below match data/stats.csv from the preprocessing step.

| Graph | Vertices | Directed arcs | Edges (arcs / 2) | Average degree | Maximum degree |
|---|---|---|---|---|---|
| Full (cleaned) | 2,758,119 | 6,794,808 | - | 2.464 | - |
| 1,000 | 1,000 | 2,378 | 1,189 | 2.378 | 5 |
| 5,000 | 5,000 | 12,096 | 6,048 | 2.419 | 6 |
| 10,000 | 10,000 | 24,296 | 12,148 | 2.430 | 6 |
| 25,000 | 25,000 | 61,472 | 30,736 | 2.459 | 6 |
| 50,000 | 50,000 | 124,898 | 62,449 | 2.498 | 7 |
| 100,000 | 100,000 | 253,430 | 126,715 | 2.534 | 7 |

## 3. Implementation and Experimental Setup (Parts C, D and E)

- **Dijkstra:** a binary min-heap (std::priority_queue with std::greater) written into a hand-coded search loop, lazy deletion of stale heap entries, early stop when the destination is settled, and explicit path reconstruction by following a parent array from the destination back to the source.
- **A*:** the same structure, with the heap ordered by f(v) = g(v) + h(v), ties broken towards the larger g. The heuristic is h(v) = floor(scale x haversine(v, destination)). The program computes the scale itself as the minimum of (arc weight / haversine length of the arc) over all arcs of the loaded graph, with a tiny safety margin for floating-point rounding.
- **Disconnected pairs:** if the queue empties before the destination is settled, the program reports ‘no path’ with cost -1 and continues. This was checked on a small hand-made graph with two components. The generated subgraphs are connected, so every experimental query is reachable.
- **Correctness checks:** the Part E program re-validates every returned path edge by edge (consecutive vertices are adjacent and the weights add up to the reported cost) and compares the Dijkstra and A* cost on every query.
- **Metrics:** vertices explored is the number of vertices settled (popped from the queue and not stale). Edges examined is the number of adjacency entries scanned from settled vertices. Execution time is the wall-clock time of one search including allocation of the distance and parent arrays and path reconstruction; graph loading and the one-off heuristic setup are excluded.

- **Queries:** ten source-destination pairs per size, chosen uniformly at random with a fixed seed (42 plus the graph size) so the experiments are reproducible. The complete vertex sequence of every path is stored in the paths CSV files.
- **Timing:** each search is run once as a warm-up and then 5 times; the mean of the 5 runs is reported.
- **Build:** compiled with g++ using -O2 -std=c++17. Execution times depend on the hardware, so the comparison between the two algorithms, run on the same machine, matters more than the absolute values.

## 4. Results (Part E)

Each value is the average over the ten queries of that size; times are the mean of 5 timed runs after one warm-up. Costs are in dataset weight units of about 0.1 m (Section 2), so dividing by 10,000 gives kilometres: the average path is 9.9 km at 1,000 vertices and 80.4 km at 100,000 vertices.

| Graph size | Algorithm | Avg cost | Avg vertices explored | Avg edges examined | Avg time (ms) |
|---|---|---|---|---|---|
| 1,000 | Dijkstra | 99,256.8 | 523.2 | 1,277.3 | 0.0547 |
| 1,000 | A* | 99,256.8 | 128.8 | 328.6 | 0.0228 |
| 5,000 | Dijkstra | 244,711.1 | 2,730.9 | 6,659.1 | 0.3566 |
| 5,000 | A* | 244,711.1 | 719.5 | 1,778.4 | 0.1728 |
| 10,000 | Dijkstra | 356,394.4 | 5,427.0 | 13,262.5 | 0.5431 |
| 10,000 | A* | 356,394.4 | 1,803.4 | 4,456.2 | 0.2489 |
| 25,000 | Dijkstra | 385,575.3 | 10,198.6 | 25,089.1 | 1.2585 |
| 25,000 | A* | 385,575.3 | 2,741.4 | 6,795.9 | 0.6647 |
| 50,000 | Dijkstra | 556,778.3 | 25,258.1 | 63,249.0 | 4.5639 |
| 50,000 | A* | 556,778.3 | 5,742.1 | 14,380.7 | 1.1887 |
| 100,000 | Dijkstra | 803,874.9 | 49,856.1 | 126,804.0 | 9.6712 |
| 100,000 | A* | 803,874.9 | 15,428.1 | 39,197.7 | 3.9435 |

Source: data/results/partE_table.md (ten queries per size).

**A* relative to Dijkstra**

| Graph size | Vertices explored (A*/Dijkstra) | Reduction in vertices explored | Edges examined (A*/Dijkstra) | Time speed-up (Dijkstra/A*) | A* heuristic scale |
|---|---|---|---|---|---|
| 1,000 | 0.246 | 75.4% | 0.257 | 2.40x | 9.9237 |
| 5,000 | 0.263 | 73.7% | 0.267 | 2.06x | 9.9237 |
| 10,000 | 0.332 | 66.8% | 0.336 | 2.18x | 9.6518 |
| 25,000 | 0.269 | 73.1% | 0.271 | 1.89x | 8.9932 |
| 50,000 | 0.227 | 77.3% | 0.227 | 3.84x | 8.9932 |
| 100,000 | 0.309 | 69.1% | 0.309 | 2.45x | 8.0237 |

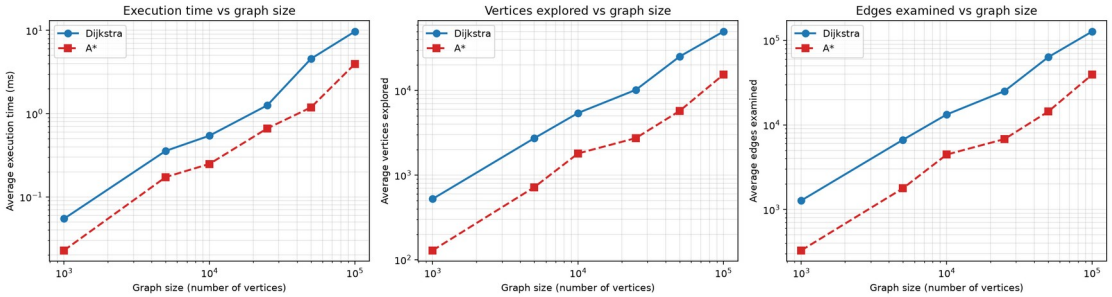



***Figure 1.*** *Average execution time, vertices explored and edges examined for Dijkstra and A* against graph size (both axes logarithmic). A* is below Dijkstra on all three measures at every size.*

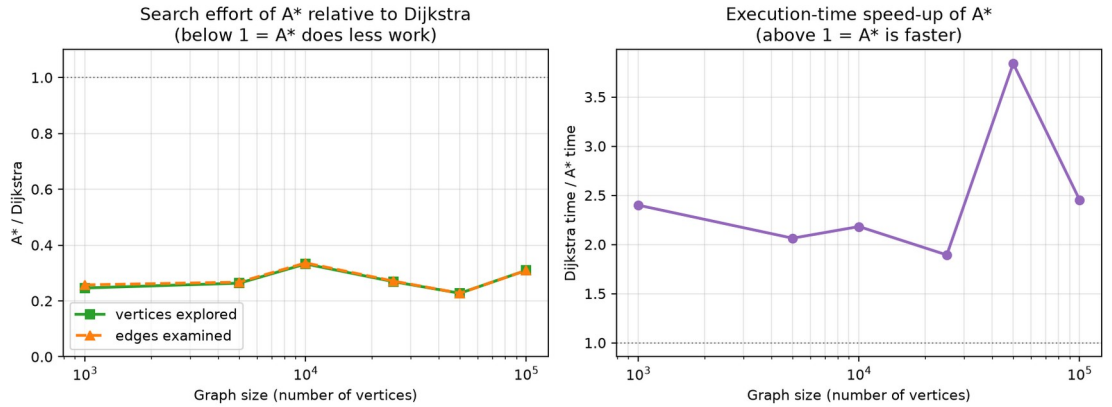



***Figure 2.*** *A* relative to Dijkstra. Left: A*/Dijkstra ratio of vertices explored and edges examined (below 1 means A* does less work). Right: execution-time speed-up, Dijkstra time divided by A* time (above 1 means A* is faster).*

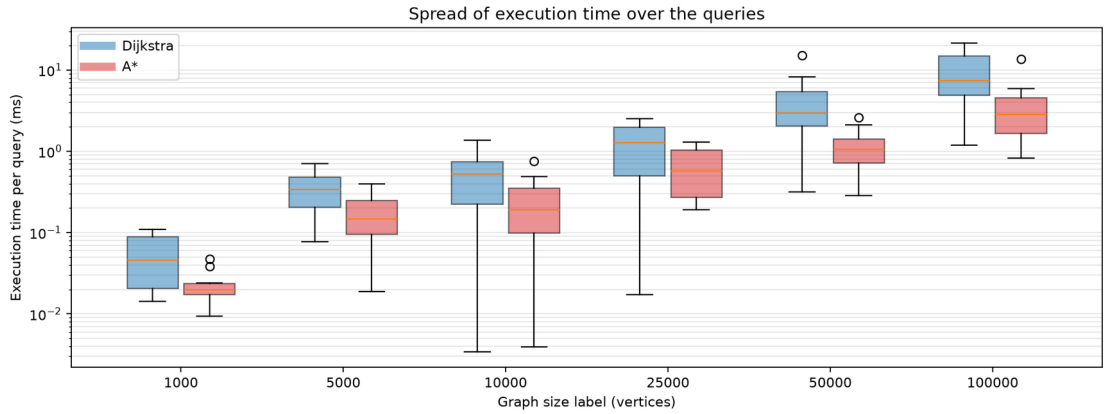



***Figure 3.*** *Spread of the execution time over the ten queries of each size (box plots, logarithmic axis). The median A* time is lower than the median Dijkstra time at every size, but the ranges overlap because the cost of a query depends strongly on how far apart its two endpoints are.*

## 5. Observations

1. **Identical optimal costs.** Dijkstra and A* give the same cost on every one of the 60 pairs, not only on average (for example 99,256.8 at 1,000 vertices and 803,874.9 at 100,000 vertices).

2. **A* explores fewer vertices.** The reduction in the average ranges from 66.8% (at 10,000 vertices) to 77.3% (at 50,000 vertices), with a mean of about 72.6%; A* settles roughly a quarter to a third of the vertices Dijkstra settles. On individual queries A* settled between 6% and 60% of Dijkstra's vertices and never more. This agrees with the theory: with a consistent heuristic A* only settles vertices whose f value is at most the optimal cost, and every such

vertex also has g at most the optimal cost, so A* settles a subset of what Dijkstra settles (up to ties at the boundary cost).

3. **Edges examined follow vertices explored.** Each settled vertex scans its outgoing arcs once, so edges examined divided by vertices explored should be close to the average out-degree. In my runs the ratio was between 2.44 and 2.55 for both algorithms, matching the graphs' average degree (2.38 to 2.53).
4. **The A* advantage does not change steadily with size.** The reduction in vertices explored is 75.4%, 73.7%, 66.8%, 73.1%, 77.3% and 69.1% for the six sizes. Each size uses its own ten random pairs and the subgraphs are nested, so these fluctuations most likely come from the particular pairs. The heuristic scale falls from 9.92 to 8.02 as the graphs grow (Q8), which makes the heuristic slightly weaker, but the effect is not visible in the reduction.
5. **Time grows with graph size.** Dijkstra's average time went from 0.0547 ms to 9.6712 ms (about 177 times) while the graph grew 100 times and the average number of vertices explored grew about 95 times. Time grew somewhat faster than the explored count; likely reasons are per-query allocation of arrays proportional to the graph size and cache misses on larger arrays and heaps (not isolated).
6. **A* is faster in wall-clock time at every size.** A* took 26% to 53% of Dijkstra's time (1.89 times faster at 25,000 vertices up to 3.84 times at 50,000), and was faster on 53 of the 60 individual queries. The gain in time is smaller than the gain in vertices explored (at 1,000 vertices: 4.1 times fewer vertices but 2.4 times faster) because each vertex A* touches needs a haversine calculation. The speed-up is consistent across all six sizes, but absolute times below 0.1 ms at the smallest sizes are close to timer resolution.
7. **Path cost grows with size.** The average path cost rose from 99,256.8 to 803,874.9 units (about 8.1 times), which is about 9.9 km to 80.4 km, because random pairs inside a larger BFS ball are further apart and the search covers a larger area.
8. **The graphs are very sparse.** Average degree is between 2.38 and 2.53 at every size, so the number of arcs is only about 2.4 to 2.5 times the number of vertices.

## 6. Answers to Part F

**Q1. Why is the dataset suitable for shortest-path analysis?**

The LKS network is real road data with 2,758,119 vertices and about 6.8 million arcs, large enough to cut out subgraphs from 1,000 to 100,000 vertices from one consistent source. Its arcs carry positive integer distance weights, as Dijkstra requires. Every vertex has latitude and longitude, which makes a geometric A* heuristic possible. It is sparse and has the structure of a real mobility network (intersections joined by road segments), so it is a realistic model for vehicle route planning. It is also public, standard and widely used in the shortest-path literature.

**Q2. How did you transform the raw dataset into a weighted graph?**

The steps are those of Section 2: parse the .gr and .co files, shift node ids to 0-indexed, convert the coordinates to degrees, remove self-loops and non-positive weights, merge parallel arcs by minimum weight, keep the largest connected component, extract BFS subgraphs of the required sizes, renumber the vertices, and write CSV files that the C++ programs load into an adjacency list. Nothing that the shortest-path computation does not need was kept.

**Q3. Do Dijkstra and A* produce the same optimal path cost in your experiments?**

**Yes.** The Part E program compares the two costs on every individual pair, not only the averages, so offsetting differences cannot hide. Across all 60 pairs (10 per size, 6 sizes) the costs were identical, and every pair was reachable. The averages agree too (for example 99,256.8 at 1,000 vertices and 803,874.9 at 100,000 vertices). This is expected: A* returns an optimal path whenever its heuristic is admissible and consistent (Q8).

**Q4. Which algorithm explores fewer vertices, and why?**

A* explores fewer vertices at every size (66.8% to 77.3% fewer on average; on individual queries it settled between 6% and 60% of Dijkstra's vertices, never more). Dijkstra orders vertices only by g(v), the distance from the source, so it expands in all directions around the source until it reaches the destination. A* orders them by f(v) = g(v) + h(v), where h(v) estimates the remaining distance, so vertices that lead away from the destination are expanded later or never. How large the saving is depends on how tight the heuristic is. The scale (8.02 to 9.92 depending on the subgraph) is set by the arc with the smallest weight-to-distance ratio, while the median ratio is 9.99, so on a typical arc the heuristic uses about 80% to 99% of the straight-line distance expressed in weight units. Real roads are not straight lines, so the heuristic cannot be exact.

**Q5. How does execution time vary as the graph size increases?**

Time grows with graph size for both algorithms: Dijkstra rises from 0.0547 ms (1,000 vertices) to 9.6712 ms (100,000 vertices), about 177 times, and A* from 0.0228 ms to 3.9435 ms, about 173 times. Over the same range the graph is 100 times larger and the average number of vertices explored grows about 95 times for Dijkstra and 120 times for A*, because random pairs in a larger graph are further apart. Time therefore grows somewhat faster than the explored count. Likely reasons are that each query allocates and initialises distance and parent arrays whose size is proportional to the graph, and that larger arrays and heaps cause more cache misses; I did not isolate these effects. Times below 0.1 ms at the smallest sizes are close to timer resolution and should be read cautiously.

**Q6. Explain the effect of the graph's density on shortest-path computation.**

Density determines how many edges are examined for each settled vertex. Here the average degree is only 2.38 to 2.53, so the number of arcs E is about 2.4 to 2.5 times the number of vertices V and the work per vertex is small and almost constant; the running time is governed by V log V. In a denser graph each settled vertex would need more edge examinations and more heap insertions, so the E log V term would dominate, and in the extreme case of E close to V squared a heap-based Dijkstra would be slower than a simple array-based version that runs in O(V squared). Because the degree barely changes between sizes (the slight rise from 2.378 to 2.534 comes from BFS subgraphs losing arcs at their boundary), these experiments isolate the effect of size and do not vary density.

**Q7. What is the theoretical time complexity of your Dijkstra implementation?**

With a binary heap and lazy deletion, every vertex is settled at most once and every arc is scanned at most once. Each successful relaxation inserts one heap entry, so the heap holds at most E + 1 entries and each insertion or removal costs O(log E), which is O(log V) since E is at most V squared. The total is O((V + E) log V), which is O(E log V) for a connected graph. Space is O(V + E) for the graph plus O(V) for the distance and parent arrays and O(E) for the heap. For this sparse graph E = O(V), so the running time is O(V log V). Because the search stops when the destination is settled, the actual work depends on the number of vertices closer to the source than the destination, which is why the measured counts are well below V.

**Q8. What heuristic did you use for A*? Is it admissible and/or consistent?**

The heuristic is h(v) = floor(scale x d(v, destination)), where d is the great-circle (haversine) distance in metres:

$a = \sin^2((\varphi_2 - \varphi_1)/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2((\lambda_2 - \lambda_1)/2)$, $d = 2R \cdot \arcsin(\sqrt{a})$, R = 6,371,008.8 m.

The factor scale is taken from the data: it is the minimum over all arcs of (arc weight) / (haversine distance of its endpoints). On the full graph this minimum is 7.18, the median ratio is 9.99, the 1st percentile 9.98 and the maximum 12.38 (data/heuristic.json), so almost every arc has a ratio close to 10 and the minimum comes from a few outlier arcs. The program computes the minimum for each loaded subgraph, so the scale is the largest value that is safe for that graph: 9.9237 for the 1,000 and

5,000-vertex graphs, 9.6518 for 10,000, 8.9932 for 25,000 and 50,000, and 8.0237 for 100,000. It falls as the graphs grow because the subgraphs are nested and the minimum over a larger set of arcs can only decrease; all of them are above the full-graph minimum because the weakest arcs lie outside these subgraphs.

**Admissible.** Every arc satisfies w(u, v) >= scale x d(u, v). Any path from v to the destination is a chain of arcs, so its cost is at least scale times the sum of the haversine lengths of its arcs. By the triangle inequality for great-circle distance that sum is at least d(v, destination), so the true remaining cost is at least scale x d(v, destination), which is at least h(v). The heuristic never overestimates.

**Consistent.** For any arc (u, v), scale x d(u, dest) <= scale x d(u, v) + scale x d(v, dest) <= w(u, v) + scale x d(v, dest). Taking the floor keeps this inequality because the weights are integers (floor(w + x) = w + floor(x)). So h(u) <= w(u, v) + h(v). Consistency means a vertex never needs to be re-expanded once it is settled, and the first time the destination is popped its cost is optimal.

**Trade-off.** The scale is set by the weakest arc. For the 100,000-vertex graph it is 8.02 against a median ratio of 9.99, so on a typical arc the heuristic is about 80% as strong as it could be (about 99% at 1,000 vertices). A larger scale would prune more, but it would overestimate on the few low-ratio arcs and optimality would no longer be guaranteed.

**Q9. What difficulties did you face while converting the real-world dataset into a graph?**

- **Choosing the dataset.** The first candidate was an online social-network message dataset. I rejected it after inspecting it: its edges are messages rather than movements, it has only about 1,900 users (too few for the 5,000 to 50,000+ vertex experiments), and it has no coordinates for an A* heuristic. The DIMACS road network meets all the requirements.
- **Two input files and unit conversions.** Arcs and coordinates are in separate files, node ids are 1-indexed, and coordinates are integers in units of $10^{-6}$ degrees.
- **Size of the full graph.** With 2.76 million vertices and 6.9 million arcs the full graph is too large for slow, unoptimised parsing, so the parsing and cleaning use NumPy arrays and only small BFS subgraphs are handed to the C++ programs.
- **Duplicate arcs.** 90,850 parallel arcs had to be merged (minimum weight kept) so that every adjacency list is clean.
- **Directed storage of undirected roads.** Each road is two arcs, so arcs, edges and degrees must be reported carefully (edges = arcs / 2).
- **Edge weight units.** The weights are not in metres: the median ratio of arc weight to haversine distance in metres is 9.99 (1st percentile 9.98), so the weights are approximately in units of 0.1 m. I determined the unit empirically from the data. A few arcs have ratios as low as 7.18 and as high as 12.38, which is why the heuristic scale must be measured from the data. All costs in this report are in these units; dividing by 10,000 gives kilometres.
- **Extracting subgraphs.** A BFS ball is a connected patch, but arcs crossing its boundary are lost, so some shortest paths of the full graph do not exist inside a subgraph, and because all sizes use one seed the subgraphs are nested rather than independent samples.
- **Portability.** The constant M_PI is not defined by MinGW g++ with -std=c++17, so the compile failed on Windows until pi was written out as a constant.

**Q10. Which algorithm would you recommend for large-scale IoV/UAV route planning?**

I would recommend A* with an admissible geometric heuristic whenever positions are available, as they are for vehicles and UAVs with GPS. In the experiments it returned exactly the same optimal costs as Dijkstra while exploring about 72.6% fewer vertices on average, and it needs no preprocessing. The saving matters most on large graphs and when many queries are answered, since each query touches fewer vertices.

The wall-clock advantage was clear: A* was 1.9 to 3.8 times faster at every size, taking 26% to 53% of Dijkstra's time. It is smaller than the reduction in explored vertices (at 1,000 vertices, 4.1 times fewer vertices but 2.4 times faster) because each vertex touched needs a haversine calculation. A cheaper heuristic, such as Euclidean distance on projected coordinates, would reduce that cost. Dijkstra remains the better choice when coordinates are missing, when one source needs distances to many destinations at once, or when edge costs have no reliable geometric lower bound.

For dynamic costs, if congestion only increases the weights, the distance-based heuristic stays admissible. If conditions can make a link cheaper than before, the heuristic scale must be recomputed from the new weights, or optimality can be lost. For networks much larger than the 100,000-vertex graphs tested here, speed-up techniques with preprocessing would be worth investigating, but I did not test them.

## 7. Limitations

- Each size uses only ten random source-destination pairs, so the averages are sensitive to the particular pairs chosen and trends between sizes should be read cautiously.
- Times are means of 5 runs after a warm-up, but sub-millisecond values are still affected by timer resolution and by the machine, so small timing differences are not reliable.
- All results come from one region (the Great Lakes) and one seed vertex for the subgraph extraction; queries lie inside a compact BFS ball, so very long routes across the whole network are not represented.
- The bonus task on dynamic edge costs was not attempted.

## 8. Reproducibility

The source code, run instructions, seeds, tool versions and the dataset citation are in the project README. The commands, in order, are: preprocess.py (parse, clean, extract subgraphs and query pairs; random seed 42), the C++ programs compiled with g++ -O2 -std=c++17 (partB_all for graph statistics, partC_dijkstra and partD_astar for single queries, partE_experiments for the full evaluation), then plot_results.py. Costs, paths, vertices explored and edges examined are deterministic; only execution times vary between machines and runs. Per-query results are in data/results/partE_per_query.csv, the averages in partE_summary.csv, and the complete paths in the *_paths.csv files.

## 9. References


1. 9th DIMACS Implementation Challenge, Shortest Paths (2005-2006). http://www.diag.uniroma1.it/challenge9/ (road network USA-road-d.LKS, from US Census Bureau TIGER/Line data).
2. C. Demetrescu, A. V. Goldberg, D. S. Johnson (eds.), The Shortest Path Problem: Ninth DIMACS Implementation Challenge, DIMACS Series in Discrete Mathematics and Theoretical Computer Science, vol. 74, American Mathematical Society, 2009.
3. E. W. Dijkstra, A note on two problems in connexion with graphs, Numerische Mathematik 1, 269-271, 1959.
4. P. E. Hart, N. J. Nilsson, B. Raphael, A formal basis for the heuristic determination of minimum cost paths, IEEE Transactions on Systems Science and Cybernetics 4(2), 100-107, 1968.